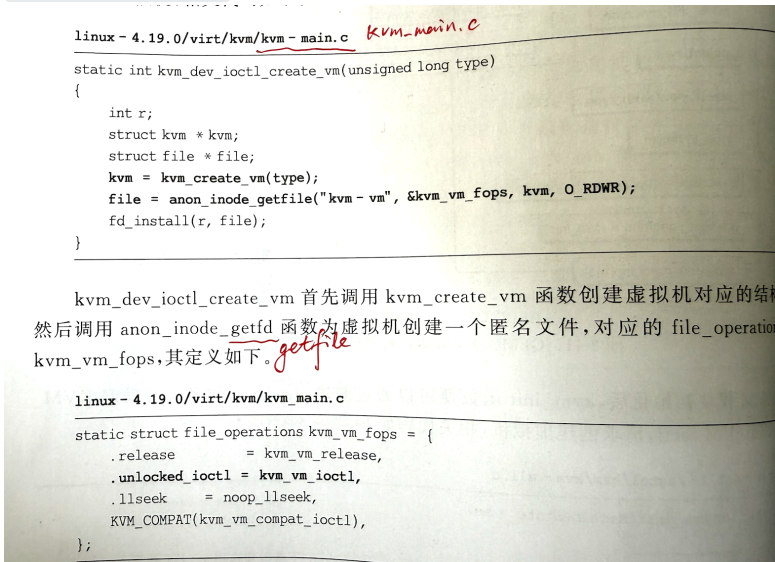


勘误表（印次：2022年8月第3次印刷）

序号	页码	描述
1	P48	文件路径 <code>kvm-main.c</code> 应改为 <code>kvm_main.c</code>；创建匿名文件的函数 <code>anon_inode_getfd</code> 应改为 <code>anon_inode_getfile</code>。  <code>linux - 4.19.0/virt/kvm/kvm-main.c</code> <i>kvm-main.c</i> <pre>static int kvm_dev_ioctl_create_vm(unsigned long type) { int r; struct kvm *kvm; struct file *file; kvm = kvm_create_vm(type); file = anon_inode_getfile("kvm-vm", &kvm_vm_fops, kvm, O_RDWR); fd_install(r, file); }</pre> <code>kvm_dev_ioctl_create_vm</code> 首先调用 <code>kvm_create_vm</code> 函数创建虚拟机对应的结构体，然后调用 <code>anon_inode_getfd</code> 函数为虚拟机创建一个匿名文件，对应的 <code>file_operations</code> 为 <code>kvm_vm_fops</code>，其定义如下。<i>getfile</i> <code>linux - 4.19.0/virt/kvm/kvm_main.c</code> <pre>static struct file_operations kvm_vm_fops = { .release = kvm_vm_release, .unlocked_ioctl = kvm_vm_ioctl, .llseek = noop_llseek, KVM_COMPAT(kvm_vm_compat_ioctl), };</pre>
2	P105	P位为1时，访问其对应虚拟地址的指令未必引起缺页异常，建议。改为，。 图 3-4 64 位系统中虚拟地址的翻译 (1) P(Present)位。PTE[0]，置为 1 时该 PTE 有效，为 0 时该 PTE 无效。任何访问该 PTE 对应的虚拟地址的指令均引起缺页异常。当物理页未分配、页表项未建立(此时 PTE 的每一位均为 0)或物理页被换出时(此时 PTE 的每一位不全为 0)，P 位为 0，物理页不存在。
3	P118	<code>MemoryRegion</code> 结构体的成员为 <code>enabled</code>，原书误写为 <code>enable</code>。 QEMU 对象模型为 MR 提供了构造函数和析构函数，分别在一个 MR 实例创建时调用。在 MR 的 <code>parent_object</code> 销毁时，就会调用 MR 的析构函数。MR 创建时调用 <code>memory_region_initfn</code> 函数初始化 MR，包括将 <code>enabled</code> 置为 <code>true</code> 和初始化 <code>ops</code>、<code>subregions</code> 链表等。调用 <code>memory_region_ref</code> 函数使 MR 的 <code>parent_obj</code> 的引用数加 1，<code>memory_region_unref</code> 函数则使 MR 的 <code>parent_obj</code> 的引用数减 1，若引用计数为 0，则会调用 <code>memory_region_finalize</code> 函数完成 MR 的析构，如果是 RAM 类型的 MR，还会释放对应的 RB。
4	P119	<code>subregions</code> 的排列顺序为优先级从大到小（即 <code>MemoryRegion.priority</code> 越大，优先级越高，越靠近链表头），原书误写为“优先级从小到大。可参考源代码的插入逻辑(Line 2401-2407)： https://github.com/qemu/qemu/blob/stable-4.1/memory.c”。 <code>ram_block</code> 的容器 MR 称为纯容器 MR，容器 MR 也可以拥有自己的 <code>MemoryRegionOps</code> 以及 <code>RAMBlock</code>。在一个容器 MR 中，<code>subregions</code> 链表上的所有子 MR 按照各自的优先级从小到大排列。<code>memory_region_add_subregion_overlap</code> 函数可以指定子 MR 的优先级，如果优先级为负，则隐藏在默认子 MR 之下，反之在上。 (2) 别名(Alias)MR 指向另一个非别名类型的 MR，QEMU 通过将多个别名 MR 指向同一个 MR 实现别名。
5	P122	footnote 中的文档链接“https://qemu.readthedocs.io/en/latest/devel/memory.html”已失效，建议更新为“https://qemu.readthedocs.io/en/master/devel/memory.html”。

6	P141	KVM中保存客户虚拟机内存线性视图的结构体应为 <code>kvm_memory_slot</code>，原书误写为 <code>kvm_memory_region</code>。 客户机内存线性视图的结构是 <code>kvm_memory_region</code>，用户态 QEMU 传来的数据需要转化为该数据结构进行保存，定义如下。 <pre>linux - 4.19.0/include/linux/kvm_host.h struct kvm_memory_slot {</pre> <i>CPA</i> <i>PAGE.SIZE</i> <i>guest frame number</i>
7	P141	<code>kvm_main.c</code> 代码片段中的 <code>new</code> 和 <code>old</code> 变量类型应为 <code>struct kvm_memory_slot</code>，原书误写为 <code>struct kvm_memory_region</code>。可参考源代码源代码(Line 920): https://elixir.bootlin.com/linux/v4.19/source/virt/kvm/kvm_main.c <pre>linux - 4.19.0/virt/kvm/kvm_main.c int __kvm_set_memory_region(struct kvm *kvm, const struct kvm_userspace_memory_region *mem) { struct kvm_memory_region old, new; struct kvm_memslots *slots = NULL; // 新的 memslots enum kvm_mr_change change; // 获取原位上的旧 slot new = old = *slot;</pre>
8	P145	最上面的结构体成员属于 <code>struct kvm_arch</code>，原书将这些变量误写为 <code>struct kvm_vcpu_arch</code> 的成员。可参考源代码(Line 793-795, 979-802): https://elixir.bootlin.com/linux/v4.19/source/arch/x86/include/asm/kvm_host.h <pre>unsigned int n_used_mmu_pages, n_requested_mmu_pages, n_max_mmu_pages; // 维护此 struct kvm 的所有 EPT 页表页 struct hlist_head mmu_page_hash[KVM_NUM_MMU_PAGES]; // EPT 页表页哈希表 struct list_head active_mmu_pages; // 全部活跃的 EPT 页表页 unsigned long mmu_valid_gen; // 当前版本号 }</pre>